Scaling Granite Code Models to 128K Context

Matt Stallone Vaibhav Saxena Leonid Karlinsky Bridget McGinn Tim Bula
Mayank Mishra Adriana Meza Soria Gaoyuan Zhang Aditya Prasad Yikang Shen
Saptha Surendran Shanmukha Guttula Hima Patel Parameswaran Selvam
Xuan-Hong Dang Yan Koyfman Atin Sood Rogerio Feris
Nirmit Desai David D. Cox Ruchir Puri† Rameswar Panda†

IBM Research
†Corresponding Authors
ruchir@us.ibm.com, rpanda@ibm.com

arXiv:2407.13739v1 [cs.AI] 18 Jul 2024

Abstract

This paper introduces long-context Granite code models that support effective context windows of up to 128K tokens. Our solution for scaling context length of Granite 3B/8B code models from 2K/4K to 128K consists of a light-weight continual pretraining by gradually increasing its RoPE base frequency with repository-level file packing and length-upsampled long-context data. Additionally, we also release instruction-tuned models with long-context support which are derived by further finetuning the long context base models on a mix of permissively licensed short and long-context instruction-response pairs. While comparing to the original short-context Granite code models, our long-context models achieve significant improvements on long-context tasks without any noticeable performance degradation on regular code completion benchmarks (e.g., HumanEval). We release all our long-context Granite Code models under an Apache 2.0 license for both research and commercial use.

https://github.com/ibm-granite/granite-code-models

1 Introduction

With the emergence and development of repository-level coding tasks (Liu et al., 2024; 2023b) and software development agents (OpenDevin Team, 2024), long context length becomes an important feature for code language models. While many proprietary large language models, like GPT4, Gemini, and Claude, support very long context windows, most open-source code language models could only provide relatively short context windows (CodeGemma Team et al., 2024; Rozière et al., 2023). This short context length limits the practicality of open-source code language models in real-world software development.

In this paper, we introduce the long-context Granite code 3B and 8B, a series of code language models that support effective context lengths up to 128K tokens. To achieve the extended context length, we first continue pretrain Granite Code 3B/8B base models with a repository-level code corpus and upsample the longer context repositories. Then, we instruction tune the continue pretrained model on a combination of short and long context instruction data. Due to the lack of long context instruction data, we generate multi-turn instruction data from repository-level file-packed documents with our original Granite-8B-Code-Instruct model to avoid the dependency on an existing long context model. More details of long context extension can be found in Section 2.

To evaluate the ability of long-context Granite Code models, we conduct extensive experiments on both short and long-context tasks, including HumanEvalPack, Long Code Completion, RepoBench-P, RepoQA, and Key Retrieval. Experiment results show that our long-context models significantly improve long-context performances without noticeable degradation in short-context performances. We open-source all our long-context Granite Code models under an Apache 2.0 license for research and commercial use.

2 Long Context Modeling

Our solution for scaling context length of Granite code models consists of a continual pretraining and an instruction tuning phase. Similar to prior works (Fu et al., 2024), we hold the basic hypothesis that the ability to utilize information at arbitrary input locations, is a capability that is mostly already acquired through large-scale pretraining, and that this capability can be readily extended to contexts substantially longer than seen during original pretraining (e.g., 4K to 128K) through lightweight training on appropriate data mixture.

2.1 Continual Pretraining

We continue pretrain the full attention Granite code base models using sequence parallelism[1] (Li et al., 2021) by gradually increasing its RoPE base frequency without using any sparse or linear attention. Specifically, we continue pretrain Granite Code 3B/8B base models using the original pretraining data used in Mishra et al. (2024) but with repository-level file packing and per-language context length upsampling, that we found to be critical for long-context continual pretraining. This continued training stage focused on a curated selection of programming languages, such as Python, C, C++, Go, Java, JavaScript, and TypeScript, as in Pinnaparaju et al. (2024).

To create long-context data, we develop a new approach that packs files from the same repository together, arranging them to prioritize semantic dependencies. We identify these dependencies by analyzing file imports and create a directed acyclic graph, where each file is a node and edges represent API imports between files. After breaking any cycles in the graph, we perform a topological sort to establish an ordering of files based on their semantic dependencies. We then organize the files in a repository by placing documentation and build files first, followed by the ordered set of files with semantic dependencies, and finally the remaining non-connected files. These non-connected files are arranged according to their folder structure, using a depth-first search to traverse the repository. Finally, we determine the dominant programming language of a repository based on file extensions and presence of build files, to organise repo-ordered files by programming languages.

The documents' lengths and their source domains/languages are two closely related confounding factors in data engineering because long data usually come from particular sources. Thus, in addition to repository-level file packing, we artificially oversampled longer document sequences on a per-language basis to ensure the quantity of long sequences, thereby improving the overall quality of our training data corpus, as in Fu et al. (2024); Yu (2023). In particular, we downsample documents under 4096 tokens to a rate of 10%, which we find to ensure a sufficient number of total tokens and documents. The total number of documents within the training corpus after processing is 173,336 with a mean length of 73,451.

We adjust the RoPE base frequency, introduced in Xiong et al. (2023), to support long context windows up to 128K where the base model itself is trained on 2K/4K context length. For training, we adopt a progressive approach where we doubled the context window until it reached the desired length of 128K. We train for 500 steps with a batch size of 32 and search for the optimal RoPE theta and learning rate for each iteration. For RoPE theta, we finf optimal values of 100K, 250K, 500K, 2M, and 10M for context windows of 8K, 16K, 32K, 64K, and 128K, respectively. We train with data parallelism and Flash Attention 2 until 64K tokens and then used Ring Attention (Liu et al., 2023a) to reach 128K tokens. The final models are trained for an extra 4B tokens which is only 0.1% of original pretraining data.

2.2 Instruction Tuning

Our training data for long context instruct models consists of a combination of permissively licensed data used in training the original Granite code instruct models (Mishra et al., 2024), in addition to synthetically generated code instruction datasets tailored for solving long context problems. Specifically, the 128K long context instruct models are derived by further finetuning the long context base models on a mix of short and long context data as follows.

[1] https://github.com/jzhang38/EasyContext

Short-Context Instruction Data. Our short context instruction data consists of a combination of CommitPackFT (Muennighoff et al., 2023), MathInstruct[2] (Yue et al., 2023), MetaMathQA (Yu et al., 2023), Glaive-Code-Assistant-v3[3], Self-OSS-Instruct-SC2[4], Glaive-Function-Calling-v2[5], NL2SQL[6], HelpSteer (Wang et al., 2023b), OpenPlatypus[7] (Lee et al., 2023), and a few synthetically generated datasets for API calling (Basu et al., 2024), and multi-turn code interactions with execution feedback.

Long-Context Instruction Data. The long context instruction data was synthetically generated by bootstrapping the pretraining data. For each repository-level file-packed document, we created a multi-turn dataset where the instructions within each sample were human-designed for the purpose of enhancing the long-context performance in specific tasks like generation, retrieval and translation. The responses were either parsed semantically from the original document or generated using Granite-8b-Code-Instruct-4K. The dataset first parses the document into classes, methods, and stand-alone functions. It then requests and extracts the implementations of a random subset of the extracted functions/methods (up to 5 per file in the document) and then asks for an explanation of that implementation using available documentation. Additionally, it generates instructions for implementing the sampled functions (methods) based on the remaining documentation and code with the function excluded. These questions and instructions were repeated for different functions until the desired length was achieved.

By exposing the model to both short and long context data, we aim to enhance its long context capability without sacrificing code generation performance at short input context. For finetuning, we use a multiturn loss mask for each sample, as in Wang et al. (2023a). This is particularly important as our finetuning data corpus consists of instruction-response pairs with multiple turns. However, when composing a sequence, we append an EOS token after each response from the model to prevent runaway generation during inference. We followed the same training parameters that produced our previous short-context instruct models (Mishra et al., 2024): 128 global batch size, 2e-5 learning rate, a noise multiplier of 5 for input embeddings, and padding-free transformers.

3 Results

We evaluate our long-context Granite code models on a wide variety of benchmarks by measuring key retrieval accuracy and performance during generation on code completion tasks at both short and long-context length as follows.

3.1 Benchmarks

Long Code Completion. Long Code Completion (LCC) (Guo et al., 2023) tests a model's ability to predict the next line of code from long repository-based context for Python, Java, and C#. While the benchmark's context length spans 1/2K through 8K+ tokens, it is heavily weighted around 2K tokens. Thus, following Bai et al. (2024) and Rozière et al. (2023), we rebalance this dataset for equal representation with each context length bucket (<4K, 2 – 4K, 4 – 8K, 8K+), where each bucket has 100 samples when possible.

RepoBench-P. Like LCC, RepoBench-P (Liu et al., 2023c) tests the model's next line code completion ability for long-context input. We follow the methodology in (Bai et al., 2024) by selecting the Cross-File-First data but then we rebalance the buckets based on the Starcoder tokenizer used for training out Granite code models.

RepoQA. RepoQA (Liu et al., 2024) is an advanced Needle-in-the-Haystack test that focuses on testing LLMs' capabilities on long-context code understanding and retrieval. Specifically,

[2]We removed GSM8K-RFT and Camel-Math from MathInstruct due to unknown or NC license.
[3]https://huggingface.co/datasets/glaiveai/glaive-code-assistant-v3
[4]https://huggingface.co/datasets/bigcode/self-oss-instruct-sc2-exec-filter-50k
[5]https://huggingface.co/datasets/glaiveai/glaive-function-calling-v2
[6]https://huggingface.co/datasets/bugdaryan/sql-create-context-instruction
[7]https://huggingface.co/datasets/garage-bAInd/Open-Platypus

Table 1: Exact Match (EM) performance on Long Code Completion (LCC) benchmark (Balanced). Long-context Granite code models consistently outperforms original base models at different input context from 4K to 32K.

Model	4K EM	8K EM	16K EM	32K EM
Granite-3b-Code-Base-2K	24.5	15.4	11.4	10.0
Granite-3b-Code-Base-128K	54.6	56.8	52.2	57.8
Absolute Gap	+ 30.1	+ 41.4	+ 40.8	+ 47.8
Granite-8b-Code-Base-4K	41.9	23.7	19.1	15.0
Granite-8b-Code-Base-128K	56.5	60.1	51.8	57.4
Absolute Gap	+ 14.6	+ 36.4	+ 32.7	+ 42.4

Table 2: Exact Match (EM) scores on RepoBench-P (Balanced) benchmark.

Model	4K EM	8K EM	16K EM	32K EM
Granite-3b-Code-Base-2K	22.0	17.9	15.4	14.0
Granite-3b-Code-Base-128K	39.8	46.8	43.1	45.3
Absolute Gap	+ 17.8	+ 28.9	+ 27.7	+ 31.3
Granite-8b-Code-Base-4K	27.9	23.0	15.7	7.8
Granite-8b-Code-Base-128K	42.7	44.0	44.8	44.5
Absolute Gap	+ 14.8	+ 21.0	+ 29.1	+ 36.7

given a long chunk of source code and a precise function description, and the model is asked to find the function in the context that corresponds to the description. This benchmark focuses on retrieving 10 needle functions from each of 5 languages x 10 repositories (500 sub-tasks/tests) with a set context size of 16K tokens.

Key Retrieval. This is a synthetic benchmark that tests the model's ability to find and execute a Python function buried within high-quality, syntactically correct Python code. As proposed in Rozière et al. (2023), we took the Code Contest finetuning dataset from Li et al. (2022) and concatenated Python solutions around the key function. We then asked the model to return the output of the key function by emulating a Python interpreter shell. We created sequences of lengths of 512 tokens and key offsets of 512 tokens.

HumanEvalPack. To evaluate model performance at short-context length, we adopt HumanEvalPack (Muennighoff et al., 2023), which extends Python problems of Humaneval Benchmark to five additional commonly used programming languages, namely JavaScript, Java, Go, C++, Rust to test three coding tasks (generation, explanation and fixing). We evaluate our long-context models in a zero-shot manner using greedy decoding with completion format for the base models, and with instruction template for the instruction-tuned models.

3.2 Base Model Evaluations

Table 1 and Table 2 show the results of Granite 3B/8B code models before and after long-context extension on LCC and RepoBench-P benchmarks respectively. Prior Granite code models with 2K/4K support fail to generate meaningful completions on long sequences. On the other hand, across all the context length (4K to 32K), models scaled to handle long contexts up to 128K achieve significantly higher performance. This demonstrates that long contexts are informative for code completion, and long-context Granite code models are able to effectively leverage this information to improve their generations on both benchmarks.

In Table 3, we compare the performance of Granite code base models to their counterparts prior to long-context extension. Our long-context models exhibit strong retrieval performance across different matching thresholds, while the short context versions mostly fail in finding the needle function successfully. The absolute differences averaged over 5 programming languages are very significant, e.g., +38.6% for Granite 8B model with a

Table 3: Retrieval accuracy (%) of Granite code base models on RepoQA benchmark evaluated using 16K context length at multiple thresholds of match similarity. All models are evaluated using greedy decoding with 256 new token limit.

Threshold	**0.0**	**0.1**	**0.2**	**0.3**	**0.4**	**0.5**	**0.6**	**0.7**	**0.8**	**0.9**	**1.0**
						Granite-3b-Code-Base-2K					
Python	6.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
C++	6.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
Java	4.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
TypeScript	7.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
Rust	1.5	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
Average	4.9	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
						Granite-3b-Code-Base-128K					
Python	76.0	57.0	54.0	49.0	44.0	40.0	34.0	30.0	28.0	25.0	20.0
C++	58.0	48.0	44.0	41.0	39.0	36.0	33.0	31.0	30.0	24.0	17.0
Java	59.0	50.0	44.0	42.0	40.0	37.0	35.0	31.0	26.0	20.0	16.0
TypeScript	58.0	38.0	34.0	33.0	29.0	27.0	23.0	23.0	23.0	16.0	7.0
Rust	57.0	38.0	36.0	32.0	30.0	29.0	28.0	24.0	24.0	19.0	16.0
Average	61.6	46.2	42.4	39.4	36.4	33.8	30.6	27.8	26.2	20.8	15.2
Absolute Gap	+ 56.7	+ 46.2	+ 42.4	+ 39.4	+ 36.4	+ 33.8	+ 30.6	+ 27.8	+ 26.2	+ 20.8	+ 15.2
						Granite-8b-Code-Base-4K					
Python	9.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0
C++	10.0	2.0	2.0	2.0	2.0	2.0	2.0	2.0	2.0	1.0	1.0
Java	11.0	1.0	1.0	1.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0
TypeScript	9.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
Rust	11.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
Average	10.0	0.8	0.8	0.8	0.8	0.6	0.6	0.6	0.6	0.4	0.4
						Granite-8b-Code-Base-128K					
Python	85.0	73.0	69.0	68.0	66.0	65.0	62.0	58.0	54.0	51.0	45.0
C++	60.0	45.0	42.0	40.0	37.0	35.0	35.0	34.0	32.0	27.0	23.0
Java	57.0	52.0	48.0	44.0	42.0	39.0	38.0	36.0	32.0	28.0	23.0
Typescript	64.0	55.0	49.0	48.0	44.0	40.0	38.0	36.0	35.0	28.0	12.0
Rust	74.0	67.0	65.0	59.0	57.0	54.0	51.0	46.0	43.0	38.0	31.0
Average	68.0	58.4	54.6	51.8	49.2	46.6	44.8	42.0	39.2	34.4	26.8
Absolute Gap	+ 58.0	+ 57.6	+ 54.8	+ 51.0	+ 48.6	+46.0	+ 44.2	+ 41.4	+ 38.6	+ 34.0	+ 26.4

matching threshold of 0.8. By looking at the score distribution across different programming languages, we can see that both models are doing best at Python, with 8B model consistently outperforming the 3B model. This result shows that our long-context Granite code models can better understand natural language description before retrieval, which aligns with the use of advanced code search in many practical situations.

3.3 Instruct Model Evaluations

Table 4 compares the performance of long-context instruct models to their short-context counterparts on RepoQA benchmark. As can be seen, our long-context instruct models significantly outperforms short-context versions on all 5 programming languages across different similarity thresholds. As an illustration, figure 1 demonstrates the difference between short and long-context models at similarity threshold of 0.5, where the performance of both 3B and 8B instruct models with 2K/4K context length support fails to achieve a retrieval accuracy of more than 2% across 5 languages (on average 0.6% vs 61.6% for 8B instruct model). We attribute the improvements to the knowledge learned from newly introduced synthetic long data for instruction tuning.

In Figure 2, we investigate key retrieval performance of our long-context instruct models on a synthetic benchmark built on top of Python solutions around a key function from Code Contest finetuning dataset (Li et al., 2022). Note that this retrieval task is analogous to the famous famous Needle-in-a-Haystack test, albeit tailored to code models. As can be seen from Figure 2, our 8B instruct model before long-context extension only exhibit strong retrieval performance up to 4K length, i.e., on the sequence length they were originally trained on.

Table 4: Retrieval accuracy (%) of Granite code instruct models on RepoQA benchmark at different matching thresholds (larger represent closer to exact match).

Threshold	**0.0**	**0.1**	**0.2**	**0.3**	**0.4**	**0.5**	**0.6**	**0.7**	**0.8**	**0.9**	**1.0**
Granite-3b-Instruct-Base-2K											
Python	15.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
C++	10.0	1.0	1.0	1.0	1.0	1.0	0.0	0.0	0.0	0.0	0.0
Java	8.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
TypeScript	11.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
Rust	9.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
Average	10.6	0.2	0.2	0.2	0.2	0.2	0.0	0.0	0.0	0.0	0.0
Granite-3b-Code-Instruct-128K											
Python	76.0	60.0	55.0	54.0	50.0	48.0	42.0	41.0	40.0	38.0	33.0
C++	58.0	48.0	44.0	41.0	39.0	36.0	33.0	31.0	30.0	24.0	17.0
Java	59.0	51.0	43.0	42.0	40.0	38.0	35.0	31.0	26.0	21.0	19.0
TypeScript	80.0	68.0	54.0	50.0	43.0	39.0	36.0	35.0	29.0	20.0	9.0
Rust	67.0	44.0	36.0	33.0	32.0	29.0	28.0	26.0	24.0	20.0	16.0
Average	68.0	54.0	46.4	44.0	42.6	38.0	34.8	32.8	29.8	24.6	18.8
Absolute Gap	+ 77.4	+ 53.8	+ 46.2	+ 43.8	+ 42.4	+ 37.8	+ 34.8	+ 32.8	+ 29.8	+ 24.6	+ 18.8
Granite-8b-Code-Instruct-4K											
Python	3.0	2.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
C++	10.0	2.0	2.0	2.0	2.0	2.0	2.0	2.0	2.0	2.0	1.0
Java	8.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	0.
TypeScript	10.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
Rust	4.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
Average	7.0	1.0	0.8	0.6	0.6	0.6	0.6	0.6	0.6	0.6	0.2
Granite-8b-Code-Instruct-128K											
Python	89.0	83.0	81.0	79.0	76.0	73.0	67.0	63.0	58.0	52.0	48.0
C++	63.0	51.0	46.0	42.0	41.0	37.0	36.0	30.0	24.0	15.0	3.0
Java	91.0	84.0	79.0	77.0	76.0	73.0	69.0	66.0	63.0	46.0	39.0
TypeScript	86.0	84.0	80.0	72.0	68.0	62.0	56.0	49.0	40.0	25.0	11.0
Rust	83.0	78.0	73.0	67.0	65.0	63.0	60.0	55.0	53.0	48.0	40.0
Average	82.4	76.0	71.8	67.4	65.2	61.6	57.6	52.6	47.6	37.2	28.2
Absolute Gap	+ 75.4	+ 75.0	+ 71.0	+ 66.8	+ 64.6	+ 61.0	+ 57.0	+ 52.0	+ 47.0	+ 36.6	+ 28.0

On the other hand, our context scaling demonstrates a perfect-all-green performance though we tend to view that this level of retrieval is relatively easy for long-context code LLMs.

3.4 Short Context Evaluations

While our long-context models are very effective on long sequences, we observe that our long-context scaling does not significantly change the short-context generic capability on standard code synthesis benchmarks consisting of short sequences. Table 5 summarizes the results on HumanEvalPack, where we find only an average $\sim$1% degradation for the pass@1 metric on 3B and 8B models respectively. We also test the HumanEval-Python performance in Figure 3 and observe that long context extension has any noticeable performance degradation. Interestingly, we notice improvements in HumanEval performance of long-context instruct models, which we attribute to our new long-context synthetic data added to instruction tuning. To summarize, while long-context extension comes at a minimal cost for short sequences, we believe this cost is more than offset by the potential of handling long sequences for many real downstream applications.

4 Conclusion

We present long-context Granite code models (3B and 8B) that support effective context lengths up to 128K tokens. We perform long context scaling by leveraging a simple yet effective strategy consisting of a lightweight continual pretraining followed by instruction tuning on a mix of short and long-context data. Our long-context models demonstrate much superior performance compared to their short-context counterparts without significantly affecting the short-context generic capability. We believe that given our current results,

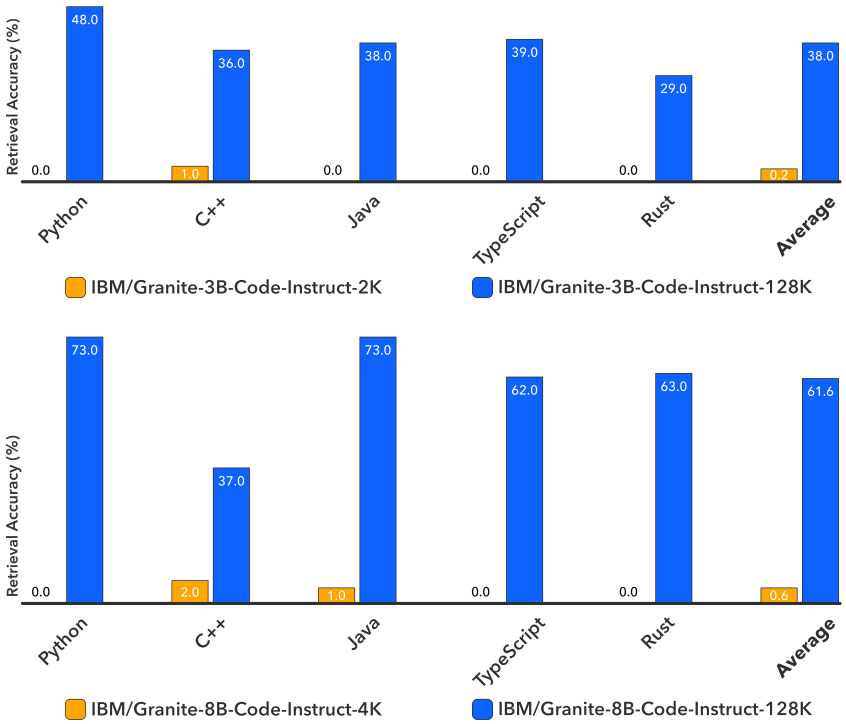

Figure 1: Retrieval accuracy of Granite 3B/8B code instruct models before and after scaling to 128K context length on RepoQA benchmark (with a matching threshold of 0.5).

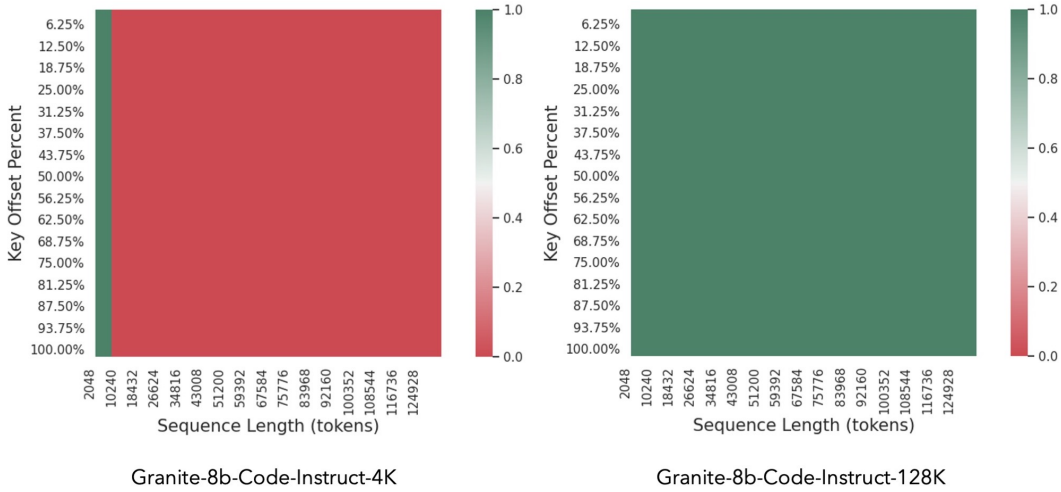

Figure 2: Key retrieval (a.k.a Needle-in-a-Haystack) performance of Granite-8B-Code-Instruct with context scaling. X-axis represents sequence length (tokens) and Y-axis represents key offset percent in retrieval. Best viewed in color.

methods to enable even longer context length and circumvent the quadratic computational complexity of attention computation will continue to further evolve (Gu & Dao, 2023). We plan to continuously release updates to these models to improve their performance and bringing the best of breed approaches to IBM Granite Family.

Table 5: Pass@1 performance on HumanEvalPack benchmark (Muennighoff et al., 2023). All models are evaluated using greedy decoding with completion format for the base models, and instruction template for the instruction-tuned models.

Model	Prompt	Synthesis	Fix	Explain	Avg.
Granite-3b-Code-Base-2K	Completion	33.0	19.5	22.2	24.9
Granite-3b-Code-Base-128K	Completion	30.5	19.9	22.4	24.2
Granite-8b-Code-Base-4K	Completion	43.1	29.1	25.4	32.5
Granite-8b-Code-Base-128K	Completion	40.2	25.2	28.2	31.2
Granite-3b-Code-Instruct-2K	Instruct	39.6	27.3	26.0	31.0
Granite-3b-Code-Instruct-128K	Instruct	41.4	26.2	25.1	30.9
Granite-8b-Code-Instruct-4K	Instruct	49.6	40.9	40.4	43.6
Granite-8b-Code-Instruct-128K	Instruct	51.4	38.3	38.9	42.9

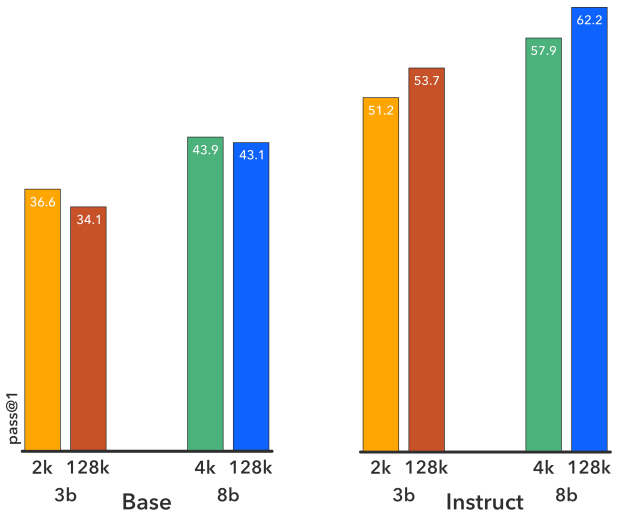

Figure 3: Effect of long-context extension on HumanEval benchmark. While we observe a slight degradation in performance for base models, instruct models see an improvement with long-context scaling, most likely due to our mixing of short-context SFT data with long-context multi-turn synthetic data. Best viewed in color.

Acknowledgments

We would like sincerely thank IBM Research leaders - Dario Gil, Sriram Raghavan, Mukesh Khare, Danny Barnett, Talia Gershon, Priya Nagpurkar, Nicholas Fuller for their support. Thanks and acknowledgement to Michele Merler, Shivdeep Singh, Manish Sethi, Pengyuan Li, Kun-Lung Wu, Syed Zawad, Andrew Coleman, Matthew White, Mark Lewis, Raju Pavuluri, Boris Lublinsky, Maximilien de Bayser, Ibrahim Abdelaziz, Kinjal Basu, Mayank Agarwal, Yi Zhou, Chris Johnson, Aanchal Goyal, Yousaf Shah, Petros Zerfos, Heiko Ludwig, Asim Munawar, Maxwell Crouse, Pavan Kapanipathi, Shweta Salaria, Bob Calio, Sophia Wen, Seetharami Seelam, Brian Belgodere, Carlos Fonseca, Colm Malone, Ray Rose, Amith Singhee, Trent Gray-Donald, Xuan Liu, Luis Angel Bathen, Abraham Daniels, Anita Govindjee, Kate Soule, and Lan Hoang.

References

Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. Longbench: A bilingual, multitask benchmark for long context understanding, 2024. URL https://arxiv.org/abs/2308.14508.

Kinjal Basu, Ibrahim Abdelaziz, Subhajit Chaudhury, Soham Dan, Maxwell Crouse, Asim Munawar, Sadhana Kumaravel, Vinod Muthusamy, Pavan Kapanipathi, and Luis A Lastras. Api-blend: A comprehensive corpora for training and benchmarking api llms. *arXiv preprint arXiv:2402.15491*, 2024.

CodeGemma Team, Ale Jakse Hartman, Andrea Hu, Christopher A. Choquette-Choo, Heri Zhao, Jane Fine, Jeffrey Hui, Jingyue Shen, Joe Kelley, Joshua Howland, Kshitij Bansal, Luke Vilnis, Mateo Wirth, Nam Nguyen, Paul Michel, Peter Choy, Pratik Joshi, Ravin Kumar, Sarmad Hashmi, Shubham Agrawal, Siqi Zuo, Tris Warkentin, and Zhitao et al. Gong. Codegemma: Open code models based on gemma. 2024. URL https://goo.gle/codegemma.

Yao Fu, Rameswar Panda, Xinyao Niu, Xiang Yue, Hannaneh Hajishirzi, Yoon Kim, and Hao Peng. Data engineering for scaling language models to 128k context. *arXiv preprint arXiv:2402.10171*, 2024.

Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.

Daya Guo, Canwen Xu, Nan Duan, Jian Yin, and Julian McAuley. Longcoder: A long-range pre-trained language model for code completion, 2023. URL https://arxiv.org/abs/2306.14893.

Ariel N. Lee, Cole J. Hunter, and Nataniel Ruiz. Platypus: Quick, cheap, and powerful refinement of llms. 2023.

Shenggui Li, Fuzhao Xue, Chaitanya Baranwal, Yongbin Li, and Yang You. Sequence parallelism: Long sequence training from system perspective. *arXiv preprint arXiv:2105.13120*, 2021.

Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d'Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. Competition-level code generation with alphacode. *Science*, 378(6624): 1092–1097, December 2022. ISSN 1095-9203. doi: 10.1126/science.abq1158. URL http://dx.doi.org/10.1126/science.abq1158.

Hao Liu, Matei Zaharia, and Pieter Abbeel. Ring attention with blockwise transformers for near-infinite context, 2023a. URL https://arxiv.org/abs/2310.01889.

Jiawei Liu, Jia Le Tian, Vijay Daita, Yuxiang Wei, Yifeng Ding, Yuhan Katherine Wang, Jun Yang, and Lingming Zhang. Repoqa: Evaluating long context code understanding. *arXiv preprint arXiv:2406.06025*, 2024.

Tianyang Liu, Canwen Xu, and Julian McAuley. Repobench: Benchmarking repository-level code auto-completion systems. *arXiv preprint arXiv:2306.03091*, 2023b.

Tianyang Liu, Canwen Xu, and Julian McAuley. Repobench: Benchmarking repository-level code auto-completion systems, 2023c. URL https://arxiv.org/abs/2306.03091.

Mayank Mishra, Matt Stallone, Gaoyuan Zhang, Yikang Shen, Aditya Prasad, Adriana Meza Soria, Michele Merler, Parameswaran Selvam, Saptha Surendran, Shivdeep Singh, et al. Granite code models: A family of open foundation models for code intelligence. *arXiv preprint arXiv:2405.04324*, 2024.

Niklas Muennighoff, Qian Liu, Armel Zebaze, Qinkai Zheng, Binyuan Hui, Terry Yue Zhuo, Swayam Singh, Xiangru Tang, Leandro von Werra, and Shayne Longpre. Octopack: Instruction tuning code large language models, 2023.

OpenDevin Team. OpenDevin: An Open Platform for AI Software Developers as Generalist Agents. https://github.com/OpenDevin/OpenDevin, 2024. Accessed: ENTER THE DATE YOU ACCESSED THE PROJECT.

Nikhil Pinnaparaju, Reshinth Adithyan, Duy Phung, Jonathan Tow, James Baicoianu, Ashish Datta, Maksym Zhuravinskyi, Dakota Mahan, Marco Bellagente, Carlos Riquelme, et al. Stable code technical report. *arXiv preprint arXiv:2404.01226*, 2024.

Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. Code llama: Open foundation models for code, 2023.

Yizhong Wang, Hamish Ivison, Pradeep Dasigi, Jack Hessel, Tushar Khot, Khyathi Raghavi Chandu, David Wadden, Kelsey MacMillan, Noah A. Smith, Iz Beltagy, and Hannaneh Hajishirzi. How far can camels go? exploring the state of instruction tuning on open resources, 2023a. URL https://arxiv.org/abs/2306.04751.

Zhilin Wang, Yi Dong, Jiaqi Zeng, Virginia Adams, Makesh Narsimhan Sreedhar, Daniel Egert, Olivier Delalleau, Jane Polak Scowcroft, Neel Kant, Aidan Swope, and Oleksii Kuchaiev. Helpsteer: Multi-attribute helpfulness dataset for steerlm, 2023b.

Wenhan Xiong, Jingyu Liu, Igor Molybog, Hejia Zhang, Prajjwal Bhargava, Rui Hou, Louis Martin, Rashi Rungta, Karthik Abinav Sankararaman, Barlas Oguz, et al. Effective long-context scaling of foundation models. *arXiv preprint arXiv:2309.16039*, 2023.

Longhui Yu, Weisen Jiang, Han Shi, Jincheng Yu, Zhengying Liu, Yu Zhang, James T Kwok, Zhenguo Li, Adrian Weller, and Weiyang Liu. Metamath: Bootstrap your own mathematical questions for large language models. *arXiv preprint arXiv:2309.12284*, 2023.

Yijiong Yu. " paraphrasing the original text" makes high accuracy long-context qa. *arXiv preprint arXiv:2312.11193*, 2023.

Xiang Yue, Xingwei Qu, Ge Zhang, Yao Fu, Wenhao Huang, Huan Sun, Yu Su, and Wenhu Chen. Mammoth: Building math generalist models through hybrid instruction tuning. *arXiv preprint arXiv:2309.05653*, 2023.